

פרק 5

הורשה ופולימורפיזם

הורשה

בדוט-נט (כמו ב-Java) אין הורשה מרובה (ב-C++ יש) 

אין רמת הרשאה בהורשה 

הגישה למחלקת הבסיס היא באמצעות base 

ניתן למנוע אפשרות ירושה ע"י sealed 

```
1. sealed class Square : Shape
2. {
3.     public Square(int x, int y)
4.         : base(x, y)
5.     {}
6.
7.     public override void Draw()
8.     {
9.         // call the Shape's Draw:
10.        base.Draw();
11.
12.        // do my own drawing here..
13.    }
14. }
```

מתודות

virtual

להכריז על מתודה שניתן לממש באופן שונה בנורש

abstract

להכריז על מתודה שחובה לממש בנורש

override

להכריז על מימוש מחדש של מתודה וירטואלית או אבסטרקטית (המימוש החדש גם הוא וירטואלי)

new

מימוש מחדש של מתודה באופן לא וירטואלי (גם אם הוכרזה כ- virtual באבא)

sealed override

מימוש מחדש של מתודה וירטואלית או אבסטרקטית, וחסמת האפשרות לדריסה וירטואלית בנורשים

new virtual

סוגי מחלקות

abstract class

- לא ניתן לייצר מופעים
- במידה ואחת המתודות אבסטרקטיות – חובה להגדיר את המחלקה כ- **abstract**

sealed class

- לא ניתן לרשת ממנה
- protected members – compiler warning
- virtual methods – compiler error

static class

- לא מכילה בנאי
- חתומה (sealed) – לא ניתן לייצר לה נורשים
- לא ניתן לייצר מופעים
- כל המתודות והשדות סטטיים (אחרת שגיאת קומפילציה)
- protected members – compiler error

המרות - casting

תחביר של המרה דומה לזה של C++

```
1. double d = 5.544
2. int a = (int)d; // a == 5
3. d = a; // back to double. d == 5.0
```

כאשר מדובר בטיפוסי ערך (Value-Types), חוקיות ההמרה נבדקת רק בזמן קומפילציה:

```
1. int i = 5;
2. bool b = (bool)i; // compile-time error!!
   "Cannot convert type 'int' to 'bool'"
3. bool b = i != 0; // the right way
```

בטיפוסי התייחסות, או שההמרה לא עוברת קומפילציה (כשאינן יחסי הורשה), או שנקבל שגיאת המרה בזמן ריצה:

```
1. B b = (B)a; // down-casting from A to B when the object is B.
2. A a1 = b; // up-casting can be done implicitly
3. X x = (X)a; // compile error!! "Cannot convert type 'A' to 'X'"
4. C c = (C)a; // run-time exception!! a is not C.
```

המרות בטוחות עם is ו-as

ניתן לבדוק יחסי הורשה וחוקיות המרה ע"י האופרטורים
:as ו-is

לבצע "המרה בטוחה" בעזרת as (שורה 1 בקוד)
המרה זו מחזירה null אם לא חוקית בזמן ריצה

לבדוק יחסי הורשה בעזרת is (שורה 7 בקוד)

```
1. C c = a as C;    // c will be null;
2. if(c != null)
3. {
4.     c.DoSomeC();
5. }

6. if(a is C)
7. {
8.     ((C)a).DoSomeC();
9. }
```


הירושה מ- Object

`string ToString()` ■

המימוש המקורי מחזיר את השם המלא של הטיפוס ■

`bool Equals(object obj)` ■

המימוש המקורי משווה התייחסויות ■

`int GetHashCode()` ■

לרוב ממומש כש – equals ממומש ■

`void Finalize()` ■

Destructor ■

`object MemberwiseClone()` ■

שיכפול רדוד ■

`Type GetType()` ■

בדיקה בזמן ריצה ■

`typeof` ■

חריגות - Exceptions

- חריגה שנזרקת "מבעבעת" מעלה בסולם הקריאות עד ש"נתפסת" (מטופלת)
- אין הכרזה על זריקת חריגה בחתימה של המתודה
- ניתן לזרוק רק אובייקטים שיורשים מ- Exception
- InnerExceptions, StackTrace, Message
- ניתן לממש כמה בלוקים של catch
- ניתן לממש catch ריק (מטפל בכל הסוגים)
- Finally
- using

ממשקים - interfaces

- במקום שבו רוצים הכרזה פולימורפית מרובה
- מתודות, מאפיינים, אירועים – הכרזה בלבד (ללא מימוש)
- מקביל למחלקה אבסטרקטית שלא מכילה מימושים

הגדרה

- לא ניתן להכיל אף מימוש, גם לא מתודות סטטיות או משלחות
- לא ניתן להגדיר שדות או קבועים
- הכל באופן אוטומטי public ו- abstract
- רמת החשיפה של הממשק עצמו – כמו שלמדנו לגבי טיפוסים

מימוש

- ניתן לרשת ממחלקה אחת ולממש כמה ממשקים שרוצים
- חובה לממש כל מה שהוכרז בממשק, גם אם באופן אבסטרקטי
- ניתן לממש באופן מפורש

structs

- מגדיר Value Type
- אין ירושה ב- struct
- אין מציין גישה protected
- יכול להכריז על מימוש ממשק/ים
- לא יכול לממש destructor
- לא יכול לממש בנאי שלא מקבל פרמטרים
- במימוש של בנאי, חובה לאתחל את כל השדות. אסור לאתחל שדה בשורת ההכרזה שלו.

פרק 6

משלחות ותכנות מונחה אירועים

שלב א' – הגדרת המשלחת

הגדרה של משלחת למתודות עם חתימה מסויימת 
במקרה הזה מתודות שמקבלות float ומחזירות bool

```
public delegate bool SomeDelegate(float f);
```